

Prism — Architecture & Sourcing Decisions

Running log of every non-obvious technical decision, why it was made, and what was verified before committing to it. Written as we go, not reconstructed at the end — timestamps are approximate build order, not wall-clock.

Hardware budget (read this first)

Target machine: RTX 5070 Laptop, **8151 MiB VRAM total**, confirmed via `nvidia-smi` at project start. Every model/infra choice below is sized against this, not against a generic "8GB card" assumption.

1. Embedding model: `nomic-embed-text` (via Ollama)

- Size: ~274MB on disk, small residual VRAM footprint when loaded alongside a quantized 7-8B generation model.
- 768-dimensional embeddings, 8k token context — comfortably covers our largest single chunk (~1,800 tokens by design, see `MAX_CHUNK_TOKENS`).
- Already available locally (`ollama list` showed it pre-installed), zero additional download cost.
- Runs through the same Ollama runtime as generation, so there's one process to manage, one health check, one deployment story — versus standing up a separate Python ML stack (sentence-transformers on CPU) purely to save the ~300MB this already-tiny model uses. If VRAM pressure becomes a real problem once the generation model is loaded, the fallback is `bge-small-en-v1.5` run on CPU via sentence-transformers — code is structured (`prism/core/embeddings.py`) so swapping the backend doesn't touch any calling code, just the `EMBED_MODEL` env var and the client call.
- **Budget math:** `nomic-embed-text`'s own weights are tiny; the real question is whether Ollama keeps both models resident simultaneously. Ollama loads models on-demand and can unload idle ones, so in the worst case embedding and generation calls interleave rather than both sitting in VRAM at full size at once. This is monitored during Phase 2 benchmarking (`eval/bench_models.py` records peak VRAM via `nvidia-smi` during a simulated retrieval+generation turn) — see Section 3 for the outcome.

2. Vector store: Chroma (embedded/persistent) + BM25 sidecar

- **Chroma**, not a client-server vector DB (Qdrant/Milvus/Weaviate server mode, Pinecone, etc.) — no separate service to run, start, or keep alive alongside Ollama on a single laptop. `PersistentClient` writes to `data/chroma/` and that's the entire deployment footprint.
- **One collection (`prism_kb`) for all five domains**, per the Phase 1 requirement: "not five separate siloed stores." Every chunk carries a domain string plus five boolean `domain_hr` / `domain_finance` / `domain_customer_support` / `domain_privacy` / `domain_legal` metadata flags (see `prism/core/store.py::_flatten_metadata`). The booleans exist because a handful of chunks are legitimately cross-tagged (e.g. warranty pages are `domain=legal` but also `also_domains=[customer_support]`), and Chroma's `where` filter needs an equality/boolean check, not a substring-contains check on a comma-joined string.

- **BM25 sidecar** (rank_bm25, pickled to data/chroma/bm25_index.pkl): added because dense embedding similarity systematically under-ranks exact token matches that these five domains actually hinge on — Kohler model numbers (κ-3901), section numbers, and Finance's numeric thresholds (■25,000, Net 45). Retrieval fuses dense + sparse results via reciprocal-rank fusion (RRF_K = 60, standard constant) rather than picking one. This is a retrieval-time fusion, not a second LLM call — no added latency/VRAM cost.
- Alternative considered: FAISS. Rejected only because Chroma's metadata filtering (where=) is more ergonomic for the multi-domain-with-cross-tags filtering this project needs; FAISS would need a hand-rolled metadata sidecar to do the same job. Both are equally valid "no server needed" choices for this hardware profile.

3. Generation model: qwen2.5:7b-instruct (locked)

Candidates benchmarked with PRISM_BENCH_LIMIT=12 on the golden set (eval/bench_models.py → eval/results/model_bench.md):

Model	Schema OK	Fact Recall	Finance Exactness	Avg Latency (s)	Peak VRAM (MB)
qwen2.5:7b-instruct	100%	74%	67%	6.4	5143
qwen3:8b	100%	74%	67%	18.4	6241
llama3.1:8b-instruct	100% 4KM	74%	33%	9.9	6022
mistral:7b-instruct	100%	78%	33%	15.4	5726

Pick: qwen2.5:7b-instruct. Reasoning tied to the 8GB constraint and the Track 3 demos:

1. **Finance exactness** (the deliberate ■-threshold demo) is 2× better than Llama/Mistral on this pass — the single most important quality signal for this submission.
2. **Qwen3 8B** (qwen3:8b, pulled and benchmarked Sep 2026) matches Qwen2.5 on schema adherence, fact recall, and finance exactness on the same 12-question slice, but is ~3× slower and uses more VRAM — not a win on this hardware profile unless a full 40-question run shows a clear gap.
3. **Latency** vs Llama/Mistral: Qwen2.5 remains fastest among models that pass the finance band task, which matters for the live demo and multi-turn UX.
4. Schema adherence is 100% across candidates after the null→[] coercion in answer.py; structured-output reliability is not a differentiator here.

Default remains overridable via PRISM_GEN_MODEL. Re-run without PRISM_BENCH_LIMIT for the full 40-question table before final PDF freeze if time allows.

Routing (separate from generation quality): on all 40 single-turn golden questions, embedding-anchor routing scored **92.1% accuracy** with **100% hit@5** and ~76ms average routing latency (eval/results/routing_smoke.json) — confirming the "no LLM call just to pick a domain"

design works.

4. Domain routing: no LLM call

Router (`prism/core/router.py`) uses cosine similarity between the query embedding and a small set of hand-written anchor phrases per domain (~12 phrases each), combined with a domain-vote over the top-10 hybrid retrieval hits. This costs one embedding call (already needed for retrieval) plus cheap vector math — zero additional LLM calls just to decide "which domain is this," per the hardware-budget instruction to reserve full LLM calls for actual reasoning/generation.

- Ambiguity margin (`ROUTER_AMBIGUITY_MARGIN = 0.04`): if the top two domain scores are within this margin, the query is treated as ambiguous and the agent asks a clarifying question instead of guessing (Piece 7, clarification-seeking).
- Confidence floor (`ROUTER_MIN_CONFIDENCE = 0.30`): below this, even the best-scoring domain isn't trusted — falls back to domain-agnostic retrieval across the whole collection.

5. Orchestration: hand-rolled state machine, not a framework

Chosen deliberately over LangGraph/LlamaIndex (see plan discussion) — the agent's control flow (route → retrieve → answer → render, with clarify/ no-answer/reformat branches) is a handful of explicit states that don't need a graph-execution framework's overhead. Every branch is a plain Python function, which makes it straightforward to unit-test each piece (routing accuracy, retrieval relevance floor, renderer correctness) independently in `eval/`.

6. Domain sourcing: 3 real, 2 synthetic — and why

Domain	Source	Why
Customer Support	Real, full scrape of <code>assist.kohler.com</code> (~360 unique articles)	Publicly available, exactly the kind of content this domain needs (troubleshooting/support), and large enough to be a genuine RAG corpus on its own.
Privacy	Real (<code>kohler.com/en/legal/privacy-policy</code> , CCPA page)	Legally required to be public; synthesizing a privacy policy would be actively counterproductive — the whole point of this domain is testing grounded, honest answers against a real legal document.

Domain	Source	Why
Legal/Compliance	Real (T&C, Prop 65, SDS, green-building disclosures, warranty pages)	Same reasoning as Privacy — Prop 65/SDS disclosures are legally mandated public documents, not goodwill content.
HR	Synthetic (data/synthetic/hr_policy.md)	No real internal Kohler HR manual is publicly available. Written from scratch, disclosed clearly (see banner at the top of the file), structurally informed by the reference BU_HR_Manual_.pdf provided for the project (a real university HR manual, used only for section/structure conventions, not content).
Finance	Synthetic (data/synthetic/finance_policy.md)	Same reasoning as HR. Structurally informed by the publicly-published AFOA "Sample Templates and Synopses of Financial Policies" reference PDF provided for the project. Deliberately contains unambiguous numeric thresholds (approval bands, per diem rates, PO requirements) as exactness-verification checkpoints for the agent — see eval/golden.jsonl.

This split is stated explicitly (not just implied by file layout) because it reflects a deliberate sourcing decision, not "grabbed whatever was easiest": Privacy and Legal *must* be real because a wrong confident answer in those domains is worse than an admitted gap, and there is no ethical way to synthesize "real" legal text. HR and Finance *must* be synthetic because no public internal Kohler document exists — but they're written to be realistic and numerically precise specifically so they can demonstrate the agent's exactness on internal-policy-style queries.

HR/Finance cross-referencing (a deliberate content design choice)

The synthetic HR policy explicitly does **not** restate monetary limits that belong to Finance (e.g. WFH equipment allowance, reimbursement thresholds) — it cross-references the Finance manual by section name instead. This prevents the two documents from silently drifting out of sync with contradictory numbers, and it doubles as a good multi-domain test case: a question like "how much is the WFH equipment allowance" should route to Finance even though WFH itself is an HR-owned policy.

7. Scraping approach: parse `__NEXT_DATA__`, not rendered HTML

`assist.kohler.com` is a Mavenoid-powered Next.js app. Every article page embeds a `<script id="__NEXT_DATA__">` JSON blob (`props.pageProps.modelSession.history[0].displayCard`) containing the exact widget content — title, markdown body, choice buttons, linked PDFs/spreadsheets — with **zero nav/footer/boilerplate**, because that content was never in the JSON to begin with. This is more reliable than scraping rendered HTML text and stripping nav/footer noise after the fact, and it was verified by hand against several article types (troubleshooting, video-only, warranty) before writing the parser — see `parse_assist.py`'s module docstring for the three shapes handled.

`kohler.com` (the parent site, hosting Privacy/Terms/Legal) is Akamai-fronted and returns 403 to requests without a full browser TLS/header fingerprint. Verified: bare `httpx/PowerShell` `Invoke-WebRequest` → 403; `curl_cffi` with `impersonate="chrome"` → 200. `fetch.py` uses `curl_cffi` specifically for `kohler.com` requests, plain `httpx` for `assist.kohler.com` (no bot-protection observed there).

Rate limiting observed in practice: repeated rapid manual test requests to `kohler.com/en/legal/*` during initial reconnaissance triggered intermittent 403/429 responses even with correct headers, confirming Akamai's protection is frequency-sensitive, not just fingerprint-sensitive. Production crawl code respects the ~1 req/sec throttle in `fetch.py` regardless of backend, which should avoid re-triggering this.

8. Scale check (the "flag early if too slow" requirement)

Verified directly against the live sitemap before writing any crawler code: **381 raw links** → **359 unique article slugs** (5 cross-listed across categories) + 1 standalone return-policy article, closely matching the brief's ~375+ estimate. At the mandated ~1 req/sec throttle, the full customer-support crawl takes **~6 minutes**. Embedding ~500-700 total chunks (customer support + legal + privacy + HR + finance) with nomic-embed-text is a matter of seconds to low minutes on this GPU. **No subset prioritization is needed — the full scrape is in scope and was completed at full size.**

One real gap found during reconnaissance: `assist.kohler.com/en/cookies` (the CCPA "Do Not Sell or Share" page linked from the sitemap and every page's footer) returned **404** during initial checks. Per the "real documents only, do not synthesize" rule for Privacy, this is **flagged in the crawl report** (`data/processed/_crawl_legal_report.json`) rather than silently dropped or faked — if it's still unreachable at final crawl time, the Privacy domain relies on the main Privacy Policy page alone, which does itself describe CCPA rights.

9. PDF rendering: `xhtml2pdf`, not WeasyPrint

WeasyPrint needs a GTK3/Cairo/Pango native runtime that isn't reliably installable on Windows without extra system setup. `xhtml2pdf` is pure Python (ReportLab-based) and renders the Markdown→HTML→PDF pipeline needed for the synthetic HR/Finance documents (and later, the prompts-documentation and deck PDFs) without any native dependency risk on the target machine.